Phase Two Delivered! Thanks to Apache XMLBeans

In this article, the author shares his experience about how the use of Apache XMLBeans helped him to overcome many development issues and deliver his project without hiccups.

Starting on a new project is usually a thrilling experience as well as a platform for learning. But when the bugs start rolling in, the enthusiasm starts waning pretty quickly. I had decided to join a project in my organisation, and was designated to take over as the project manager. As a run-up to joining the team, I was informed that the project was in Phase One of development and would be delivered shortly for testing. In fact, Phase One code was delivered for testing just a few days after I joined the team. But the delivered code had a lot of bugs, which resulted in an escalation to seniors. Though I was new in the team, as a senior, I had to take responsibility for the execution of the project and to ensure its smooth delivery.

Phase One of the project was supposed to be straightforward, but the task became complicated when the team decided to implement a custom XML parser. Today, the task of reading an XML file appears simple. We only have to locate the necessary libraries, and use them to read the input XML. But at that time (around 2006, to be precise), when few libraries were available, the team members had decided to implement XML parsing on their own. This decision turned into the monster that came back to bite us.

The code developed by the team had bugs, due to which the input XML was not being read properly. We spent quite a few long days in the office trying to resolve the issues and get the project back on track. Finally, a few days and many frayed tempers later, we managed to complete the task of reading the XML file, followed by validation of the business logic (which, interestingly, had less bugs) and delivered Phase One for integration testing.

Apache XMLBeans

Based on the experience of delivering Phase One, I knew that I had a tough task on my hands. The reason for this anxiety was the increased complexity of the input XML in Phase Two of the project. To try and avoid the difficulties faced during Phase One, I discussed this problem with a few colleagues, and one of them suggested that I look at Apache XMLBeans. After an initial exploration of XMLBeans and trying it on a sample XML, I took it for a spin on one of the XMLs of Phase Two. I was pleasantly surprised by the ease with which I was able to parse the input XML, without any issues. Based on this experience, I trained the team to use the library for delivering Phase Two of the project.

Generating the JAR

Apache XMLBeans is a Java-based library that allows us to read XML documents. To use the library, we need to generate a JAR file corresponding to the input XML. The JAR file can be created from the XSD using the 'scomp' utility provided by the library. In case the XSD is not available, it is possible to create one from a sample input XML, either by using one of the many XML tools like XMLSpy or by using an online application like XMLGrid.

The command used to generate the JAR from the XSD is given below (path to Javac may be different):

```
scomp -compiler "c:\jdk1.8.0_92\bin\javac.exe" catalog.xsd
-out book.jar
```

On successful execution, the command creates *book.jar*,